

# Relatório Final — Computação em Nuvem

## Projeto **DijkFood** — Trabalho A2

### Objetivo 3: Camada preditiva e conversacional

Pedro Tokar — João Felipe Vilas Boas — Vitor Matheus — Anderson Falcão

## 1. Introdução

O Trabalho A2 estende a plataforma **DijkFood** desenvolvida na A1. A solução original já atendia ao núcleo operacional do sistema de delivery: cadastro de usuários, restaurantes, entregadores e itens de menu; criação e progressão de pedidos; cálculo de rotas; rastreamento de entregadores; e implantação em nuvem com serviços em ECS/Fargate, banco relacional em RDS PostgreSQL, armazenamento operacional de posições em DynamoDB e balanceamento por Application Load Balancer.

Nesta segunda etapa, a extensão foi desenhada para não transformar analytics, predição ou conversação em dependências críticas da operação. Assim, falhas nessas camadas devem degradar apenas funcionalidades auxiliares, preservando a criação de pedidos sempre que houver recursos operacionais disponíveis. Assim, a extensão foi desenhada de forma aditiva e desacoplada. Os serviços que atendem o usuário final continuam responsáveis apenas pelo caminho crítico da operação, enquanto a ingestão de eventos, o processamento analítico, o treinamento de modelos, a inferência batch, a inferência online e a interface conversacional passam a operar em uma camada separada.

A direção escolhida foi o **Objetivo 3**, que exige duas capacidades complementares: uma camada preditiva, capaz de gerar inferências a partir dos dados acumulados na camada analítica, e uma camada conversacional, capaz de responder perguntas em linguagem natural sobre o estado e o histórico da operação. Além disso, o trabalho exige uma camada analítica obrigatória, comum a todos os objetivos, responsável por persistir os eventos operacionais e alimentar um dashboard com indicadores agregados.

### 1.1. Objetivos específicos da A2

- Persistir eventos operacionais em uma camada durável, sem impactar a latência ou disponibilidade da operação.
- Construir um data lake em S3 com dados brutos, dados conformados e tabelas agregadas.
- Criar um dashboard analítico com indicadores históricos da operação.
- Produzir predições de tempo de entrega, demanda por região/horário e anomalias operacionais.
- Implementar uma camada conversacional baseada em Amazon Bedrock para perguntas em linguagem natural.
- Tratar explicitamente o ciclo de vida das capacidades preditivas e conversacionais: coleta, processamento, treinamento, implantação, monitoramento e integração.
- Aplicar o tema do seminário do grupo por meio da orquestração de pipelines com EventBridge Scheduler e AWS Step Functions.
- Preservar os requisitos funcionais da A1 e avaliar experimentalmente o impacto da extensão A2 sobre os requisitos não-funcionais, em especial disponibilidade, taxa de sucesso e latência sob carga.

## 2. Arquitetura proposta

A arquitetura da A2 mantém a base operacional construída na A1 e adiciona quatro grandes blocos: ingestão analítica, data lake, camada preditiva e camada conversacional. A operação principal continua concentrada nos serviços **core-api**, **order-service**, **tracking-service** e **routing-service**, executando em ECS/Fargate dentro de sub-redes privadas e compartilhando a leitura de um banco relacional PostgreSQL hospedado no RDS e uma tabela no DynamoDB. O ALB continua sendo o ponto único de entrada para o usuário e para o frontend React.

A extensão A2 é conectada à operação por fluxos de saída. Eventos de pedidos e transições de estado são capturados a partir do RDS PostgreSQL por meio de um padrão de **outbox transacional**. Eventos de posição dos entregadores são capturados por DynamoDB Streams. Ambos os fluxos são encaminhados para uma camada de ingestão serverless, que os entrega ao S3 via Amazon Data Firehose como eventos brutos em JSON comprimido. Em seguida, jobs do AWS Glue transformam esses eventos em tabelas curated e marts em Parquet, disponibilizadas para consulta via Athena.

A decisão arquitetural mais importante foi separar rigorosamente o plano operacional do plano analítico. A operação da A1 precisa continuar atendendo pedidos, rastreamento e cálculo de rotas independentemente do estado da camada analítica. Portanto, nenhum componente analítico consulta diretamente o RDS ou o DynamoDB de produção para responder dashboard, modelo ou pergunta conversacional. Essas leituras são feitas sobre S3/Athena, a partir de dados já replicados e processados.

#### **Separação operacional-analítica**

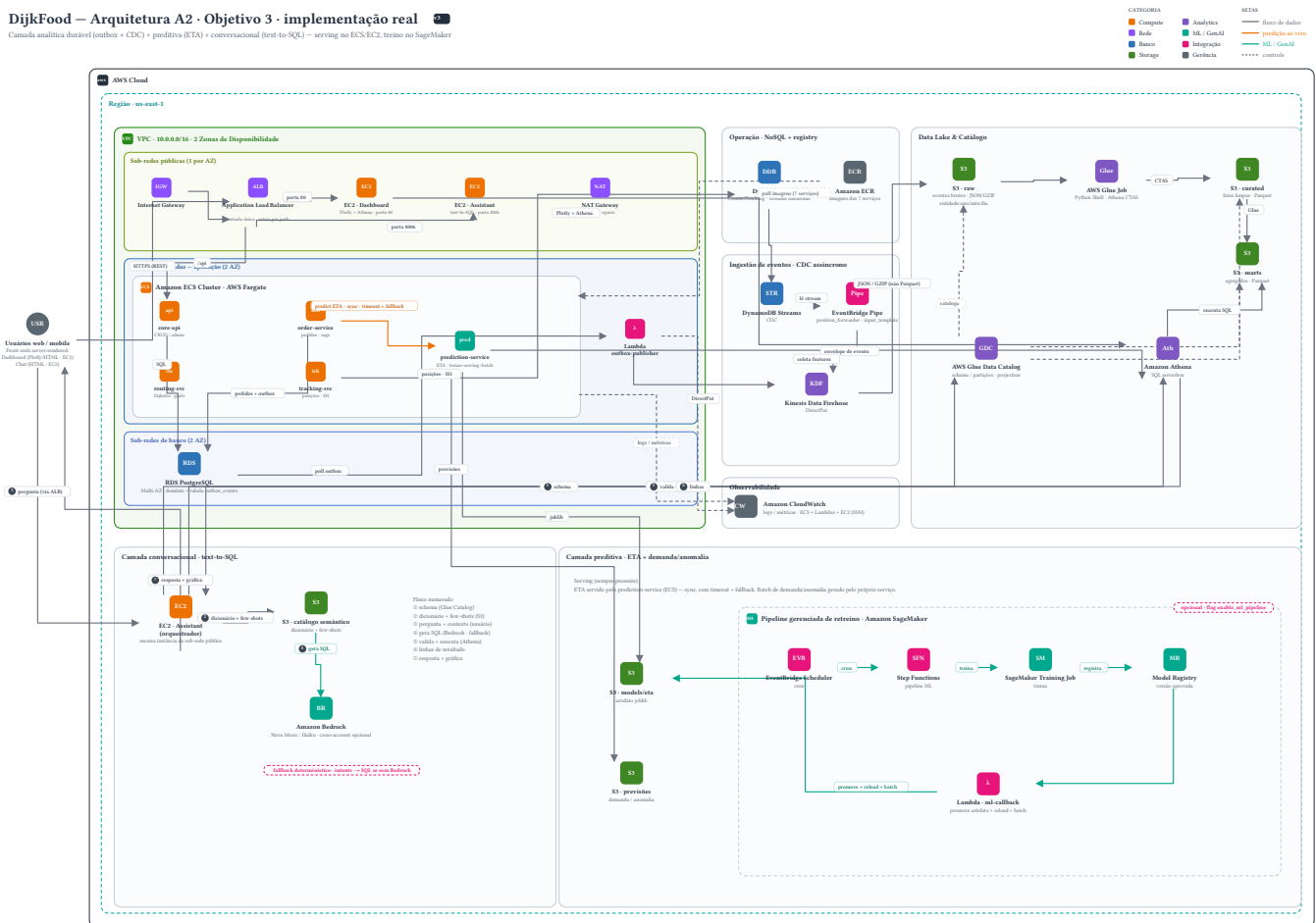
**Escolha:** captura assíncrona de eventos e leitura analítica apenas via data lake

**Justificativa:** essa escolha impede que consultas pesadas, modelos ou chamadas conversacionais concorram com o banco operacional. O RDS e o DynamoDB continuam atendendo apenas a operação transacional da A1.

## 2.1. Diagrama

DijkFood – Arquitetura A2 · Objetivo 3 · implementação real

Camada analítica durável (outbox + CDC) + preditiva (ETA) + conversacional (text-to-SQL) – serving no ECS/EC2, treino no SageMaker



## 3. Camada analítica

A camada analítica é o núcleo comum da A2. Ela recebe eventos operacionais, persiste esses eventos em armazenamento durável e produz tabelas agregadas para consulta pelo dashboard, pelos modelos preditivos e pela camada conversacional.

### 3.1. Eventos capturados

A arquitetura captura três classes de eventos:

- **Criação de pedidos:** momento em que um usuário cria um pedido, incluindo restaurante, região, entregador alocado e timestamp.
- **Transições de estado:** mudanças no ciclo de vida do pedido, como criado, confirmado, em preparo, coletado e entregue.
- **Posições reportadas:** atualizações de latitude, longitude, célula H3, status do entregador e timestamp.

Esses eventos são suficientes para reconstruir o histórico operacional da plataforma e derivar os indicadores exigidos: volume de pedidos no tempo, tempo médio em cada estado, distribuição geográfica, heatmap temporal de demanda, top restaurantes e histograma de tempo total de entrega.

### 3.2. Decisão de ingestão: outbox + DynamoDB Streams + Firehose

Foram consideradas três alternativas para a ingestão dos eventos:

| Alternativa       | Descrição                                                                         | Avaliação                                                                                                                                                 |
|-------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dual-write direto | O serviço operacional escreveria no banco e também diretamente no S3 ou Firehose. | Rejeitada por risco de inconsistência: o banco poderia confirmar e o evento falhar, ou o evento ser publicado sem a transação operacional ser confirmada. |

| Alternativa              | Descrição                                                                                            | Avaliação                                                                                     |
|--------------------------|------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Leitura periódica do RDS | Um job externo consultaria periodicamente as tabelas operacionais.                                   | Rejeitada por acoplar o analítico ao banco de produção e adicionar carga de leitura ao RDS.   |
| Outbox + streams         | Eventos de RDS são gravados em uma outbox transacional; eventos de DynamoDB são emitidos por stream. | Escolhida por preservar atomicidade, desacoplamento e baixa interferência no caminho crítico. |

No RDS, o **order-service** grava uma linha na tabela `outbox_events` dentro da mesma transação que cria o pedido ou registra uma transição de estado. Isso garante que não exista evento analítico sem operação real, nem operação real sem evento associado. Uma Lambda **outbox-publisher** consome essa tabela e publica os eventos no Firehose. No DynamoDB, a tabela `CourierTracking` emite alterações por DynamoDB Streams e ao mesmo tempo um **pipe do EventBridge** repassa o stream e envia as posições ao Data Firehose.

#### Ingestão analítica

**Escolha:** outbox transacional para RDS, DynamoDB Streams para tracking e Firehose como backbone de entrega

**Justificativa:** essa combinação preserva a atomicidade dos pedidos, captura posições de forma assíncrona e evita que os serviços operacionais precisem conhecer detalhes do data lake.

O Amazon Data Firehose foi escolhido como mecanismo de entrega para o S3 por ser serverless, exigir pouca operação e permitir buffering por tempo ou tamanho. Diferentemente de Kinesis Data Streams puro, ele não exige dimensionamento manual de shards e é suficiente para o Objetivo 3, que não demanda atualização em poucos milissegundos como no Objetivo 1.

### 3.3. Data lake, catálogo e dashboard

A arquitetura utiliza um data lake em S3 com uma organização em três zonas: **raw**, **curated** e **marts**. Essa estrutura separa a fonte bruta dos dados limpos e dos agregados de consumo. O dashboard lê das camadas `curated` e `mart`, que também podem ser usadas pelo modelo conversacional. A camada `raw` acumula os eventos brutos em JSON.

A escolha de usar eventos brutos em JSON para a camada `raw` do data lake se deu por desacoplamento dos serviços: um único Firehose recebe eventos de diversos serviços diferentes, e cada evento tem sua própria formatação. Usar a ferramenta de conversão para Parquet nativa do serviço iria requerir o uso de várias streams do Firehose diferentes, o que complicaria a arquitetura sem ganhos para o objetivo 3.

O AWS Glue exerce dois papéis. Primeiro, atua como catálogo de dados por meio do Glue Data Catalog, mantendo tabelas, colunas, tipos e partições disponíveis para Athena, Firehose e SageMaker. Segundo, executa jobs de ETL que transformam dados `raw` em `curated` (a camada `curated` segue um formato de tabelas em Parquet bem documentadas) e `curated` em `marts`. As tabelas em `marts` são pequenas, particionadas e orientadas ao consumo, reduzindo custo e latência das consultas Athena. Elas são feitas por meio de agregações que são úteis ao negócio.

### 3.4. Indicadores do dashboard

O dashboard analítico é atendido pelo **dashboard-service**, executado em uma instância EC2 dedicada (optou-se por EC2 para otimização de custo operacional frente ao ECS/Fargate). O serviço foi desenhado seguindo os princípios de uma arquitetura que seja capaz de reconciliar dados históricos consolidados com atualizações em tempo quase real, mitigando os custos de varredura (scan) do Athena quando possível.

- **Batch Layer (Histórico consolidado):** Indicadores analíticos pesados — como o tempo médio por estado, a distribuição regional, sazonalidades (heatmap), ranking de restaurantes e histograma de entregas — consomem tabelas agregadas em formato Parquet (`mart_*`). Essas tabelas são geradas de hora em hora no S3 pelo Glue Job (**build\_marts.py**), reduzindo a leitura no Athena a apenas alguns kilobytes.
- **Speed Layer (Tempo real):** Indicadores operacionais instantâneos — como entregadores ativos nos últimos minutos, pedidos em andamento e a volumetria da hora corrente — são obtidos consultando a tabela crua de eventos (`events`), limitada a uma janela de tempo curtíssima.

O backend reconcilia essas duas camadas dinamicamente em memória (como no volume de pedidos, onde o histórico consolidado é mesclado com a hora corrente da Speed Layer). Caso os agregados em Parquet ainda não tenham sido gerados pelo primeiro processamento do Glue, o serviço possui um fallback automático que realiza a query direto na tabela crua. Além das consultas no Athena, os dados de previsão de demanda e anomalias operacionais são consumidos de forma assíncrona, lendo diretamente payloads JSON salvos pelo **prediction-service** no S3 (`predictions/`).

#### Dashboard analítico

**Escolha:** dashboard-service em EC2 implementando leitura híbrida das camadas do data lake

**Justificativa:** A fim de garantir atualização em tempo quase real sem inviabilizar o custo de scan do Athena, o dashboard adota um modelo híbrido. Ele consome agregações históricas em Parquet (Batch Layer) e limita as consultas à tabela crua de eventos a uma

janela curta (Speed Layer). A execução em EC2 dedicada simplifica a infraestrutura e reduz o custo operacional para uma ferramenta de visualização de tráfego interno.

Diferente do assistente conversacional (que precisa de flexibilidade para estruturar queries complexas), o dashboard depende de agregações previsíveis. A materialização física executada pelo Glue Job (`build_marts.py`) converte o histórico JSON/GZIP bruto em tabelas analíticas limpas (`curated_*`) e agregados prontos (`mart_*`) em formato Parquet colunar de alta performance. A alternativa de utilizar um Data Warehouse tradicional dedicado (como o Amazon Redshift) foi rejeitada para evitar custos fixos e complexidade operacional contínua, priorizando uma estratégia totalmente serverless baseada em S3, Glue e Athena.

### 3.5. Evidência experimental da ingestão

Na execução experimental, a camada analítica registrou 1.785.412 eventos operacionais no período observado. Desses, 2.993 corresponderam à criação de pedidos, 10.330 a transições de estado e 1.753.089 a posições reportadas por entregadores. O intervalo entre o primeiro e o último evento registrado foi de aproximadamente 56 minutos, confirmando que o pipeline foi capaz de persistir eventos operacionais gerados durante os testes de carga.

## 4. Camada preditiva

A camada preditiva atende às três capacidades exigidas no Objetivo 3: estimativa de tempo de entrega (ETA), previsão de demanda por região/horário e detecção de anomalias operacionais. A principal decisão de arquitetura foi a separação estrita entre inferência online (em tempo real) e processamento batch (em lote), visando eficiência de custos e latência mínima.

| Capacidade                 | Modelo usado                       | Modo de inferência                       | Justificativa                                                                                                                     |
|----------------------------|------------------------------------|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Tempo de entrega (ETA)     | Random Forest                      | Inferência online em memória (ECS)       | É a única predição síncrona exigida no momento do pedido. O modelo roda em memória no contêiner ECS para garantir escalabilidade. |
| Demanda por região/horário | Seasonal-Naive (médias históricas) | Inferência pré-computada em lote (Batch) | A previsão é periódica para as próximas 24 horas. É gerada após o retreino e salva no S3.                                         |
| Anomalias operacionais     | Estatístico (Z-Score + MAD)        | Inferência pré-computada em lote (Batch) | Identifica desvios de volumetria e atrasos em janelas de 48h. Os alertas e desvios são persistidos no S3.                         |

### 4.1. Orquestração e Pipeline de Treino Gerenciada

O ciclo de vida do modelo preditivo principal é totalmente automatizado e orquestrado na nuvem sem gerar custos fixos de servidores ativos:

1. **Gatilho Agendado:** Um **EventBridge Scheduler** inicia o processo periodicamente (por padrão, uma vez ao dia).
2. **Orquestração (Step Functions):** A máquina de estados **dijkfood-eta-retrain** gerencia o fluxo de ponta a ponta. Ela provisiona e monitora um **SageMaker Training Job** temporário, que processa os dados históricos do Athena e treina a nova versão do algoritmo **Random Forest**.
3. **Callback e Publicação:** Ao finalizar o treino com sucesso, a Step Function aciona a Lambda de callback (**dijkfood-ml-callback**). Essa função registra o modelo aprovado no **SageMaker Model Registry**, copia o artefato `.joblib` final para a pasta de serving do S3 e notifica os serviços operacionais.

### 4.2. Implantação e Serving Resiliente de ETA

Diferente do design tradicional que expõe modelos por endpoints gerenciados do SageMaker (cujo custo fixo é elevado), o **prediction-service** rodando no ECS baixa o artefato `.joblib` do S3 e o mantém **em memória**. Quando o **order-service** recebe uma nova requisição de pedido, ele dispara uma chamada HTTP síncrona e paralela para o contêiner do ECS. O modelo em memória executa a inferência localmente em menos de 2 milissegundos. Essa chamada é explicitamente protegida por timeout e fallback no código da aplicação. Caso o serviço preditivo não responda a tempo, o **order-service** intercepta a falha e atribui uma estimativa de fallback fixa de segurança (por exemplo, 35 minutos), garantindo a resiliência operacional da criação de pedidos.

Regra operacional: a predição de ETA pode enriquecer o pedido, mas a indisponibilidade ou lentidão do modelo nunca impede a criação do pedido. A resiliência operacional é prioritária.

### 4.3. Previsão de Demanda e Anomalias em Lote (Batch)

A previsão de demanda e a detecção de anomalias são executadas em lote logo após a conclusão do retreino. A Lambda de callback dispara uma requisição para o endpoint `/batch/run` do contêiner do ECS, que processa a lógica de lote:

- **Previsão de Demanda:** Realiza agregações no histórico de eventos do Athena para projetar o volume esperado de pedidos por região e hora para as próximas 24 horas.
- **Detecção de Anomalias:** Varre a operação recente utilizando **Z-Score** sobre a volumetria horária para identificar picos incomuns de demanda, e calcula desvios da mediana sobre os tempos de entrega para mapear gargalos e lentidão em regiões específicas.

Ambos os resultados são consolidados em arquivos JSON estáticos no S3. Quando o dashboard solicita esses dados, o **prediction-service** apenas lê e entrega os arquivos do S3 instantaneamente, eliminando a necessidade de consultas lentas e custosas ao Athena em tempo real.

#### Inferência preditiva

**Escolha:** Modelo de ETA servido localmente em memória no ECS (Fargate) e Batch executado localmente via triggers agendados  
**Justificativa:** Evita o custo fixo de manter endpoints ativos 24/7 no SageMaker, mantendo a latência na casa dos milissegundos para o usuário e permitindo ler previsões pesadas do S3 instantaneamente no Dashboard.

## 5. Camada conversacional

A camada conversacional é implementada pelo **assistant-service**, executado como contêiner Docker em uma instância EC2 dedicada e exposto para o frontend. Essa API usa Amazon Bedrock, quando disponível, para transformar perguntas em linguagem natural em consultas SQL seguras sobre Athena. Quando o Bedrock não está disponível no ambiente de laboratório, o serviço utiliza fallback determinístico.

A decisão principal foi não usar um agente com ferramentas complexas nem uma Knowledge Base com banco vetorial sempre ligado. Para o escopo do projeto, a abordagem mais econômica e controlável é **text-to-SQL** com contexto explícito. O Bedrock não conhece os dados sozinho; uma chamada auxiliar a um modelo gera um SQL que retorna as informações úteis e necessárias para a resposta principal.

### 5.1. Como o Bedrock recebe contexto operacional

O contexto enviado ao Bedrock vem de duas fontes:

- **Glue Data Catalog:** fornece o schema das tabelas do data lake, incluindo nomes, colunas, tipos e partições.
- **Catálogo semântico em S3:** arquivo versionado e curado com descrições de negócio, significado das colunas, enums, definições de métricas e exemplos de pergunta-SQL.

O catálogo semântico é necessário porque o schema técnico não explica sozinho o significado operacional dos dados. Por exemplo, o modelo precisa saber que “tempo total de entrega” corresponde ao intervalo entre criação do pedido e entrega final; que “região” pode ser derivada de H3; e que “pedido atrasado” depende de uma regra de negócio baseada no ETA ou em percentis históricos.

| Fonte de contexto   | Conteúdo                                                                                                   |
|---------------------|------------------------------------------------------------------------------------------------------------|
| Glue Data Catalog   | Nomes de tabelas, colunas, tipos, partições e localização dos dados no S3.                                 |
| S3 semantic catalog | Descrição de métricas, significado de colunas, enums de estado, regras de negócio e exemplos pergunta-SQL. |
| Prompt template     | Instruções fixas de segurança: gerar apenas queries com SELECT, usar partições, limitar linhas.            |

### 5.2. Fluxo conversacional

O fluxo de uma pergunta segue os passos abaixo:

1. O usuário envia uma pergunta no chat do App React.
2. A **assistant-api** carrega o schema do Glue e o catálogo semântico em S3.
3. A API monta um prompt com pergunta, schema, definições de métricas e exemplos.
4. O Bedrock gera uma consulta SQL para Athena.
5. A **assistant-api** valida a consulta: apenas leitura, tabelas permitidas, LIMIT, filtros de partição e limite de custo estimado.
6. A consulta é executada no Athena.
7. O resultado tabular retorna para a **assistant-api**.
8. Opcionalmente, uma segunda chamada curta ao Bedrock transforma as linhas em resposta textual em português.
9. O App React exibe resposta, tabela e, quando aplicável, gráfico.

#### Camada conversacional

**Escolha:** assistant-api + Bedrock em modo text-to-SQL + Athena

**Justificativa:** a abordagem evita infraestrutura vetorial permanente, reduz custo de Bedrock e mantém controle sobre segurança e gasto de consultas. O Bedrock atua como gerador e explicador, mas a execução permanece validada pela aplicação.

5.3. Segurança e controle de custo das perguntas

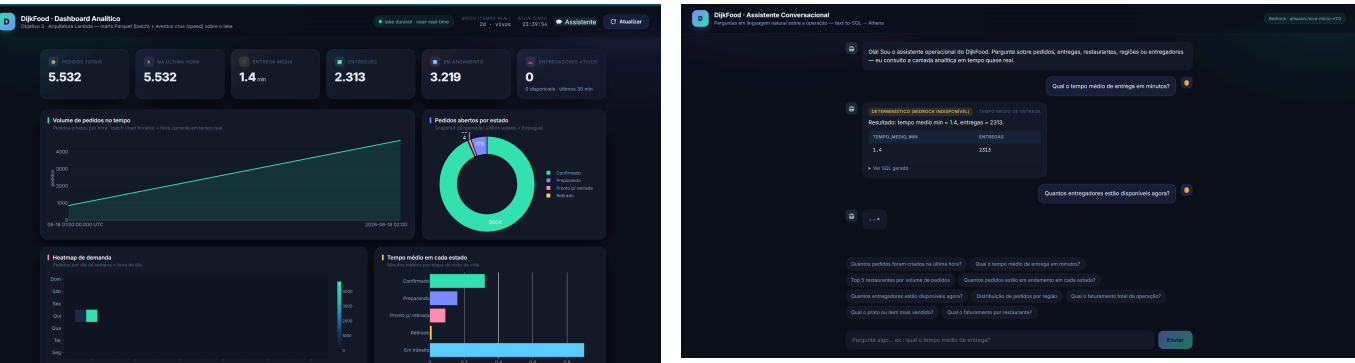
A **assistant-api** não executa SQL livre. Antes de enviar a consulta ao Athena, ela aplica regras de validação. São aceitas apenas consultas SELECT; comandos de escrita, alteração de schema ou leitura de tabelas não autorizadas são bloqueados. A API também impõe LIMIT, exige filtros de partição quando a pergunta envolve histórico amplo e rejeita consultas cujo plano indique scan excessivo.

Em caso de erro de SQL, a API pode fazer uma tentativa de autocorreção, enviando ao Bedrock a mensagem de erro do Athena junto com o schema. O número de tentativas é limitado para evitar loops e custo excessivo.

Exemplo de pergunta suportada: “Qual foi o tempo médio de entrega por região ontem?”  
A assistant-api traduz a pergunta para uma consulta sobre `mart_delivery_metrics`, valida o SQL, executa no Athena e pede ao Bedrock apenas a redação final da resposta.

5.4. Uso em conta de laboratório

O uso do serviço Bedrock não é permitido em contas de Learner Lab da AWS. Para contornar essa limitação, foi implementado um mecanismo que armazena e passa para o serviço de conversação as chaves de API para acessar o Bedrock em outra conta da AWS. Caso essa chave não seja definida, a camada conversacional sempre irá usar o fallback determinístico.



(a) **Dashboard analítico** — métricas históricas e operacionais sobre o data lake. (b) **Assistente conversacional** — perguntas em linguagem natural com consulta à camada analítica.

Figura 1: Interfaces de demonstração do Objetivo 3. À esquerda, o dashboard analítico consolida métricas de pedidos, entregas, estados, demanda temporal e entregadores. À direita, o assistente conversacional responde perguntas sobre a operação usando a camada analítica; quando o Bedrock não está disponível no ambiente de laboratório, o serviço opera em modo determinístico.

6. Segurança, tolerância a falhas e observabilidade

A arquitetura de segurança foi desenhada de forma híbrida para acomodar as restrições de rede e IAM do ambiente Learner Lab. O Application Load Balancer (ALB) é o principal ponto de entrada da aplicação operacional. Os microsserviços principais, como `order-service` e `prediction-service`, e o banco de dados RDS permanecem isolados em sub-redes privadas. Já os servidores dedicados do `dashboard-service` e do `assistant-service` residem em sub-redes públicas para simplificar a demonstração, com acesso de entrada controlado por Security Groups adequados ao ambiente de laboratório. Em produção, esses acessos deveriam ser restringidos à VPC, ao ALB ou a uma VPN administrativa.

Devido às limitações de segurança da AWS Academy (que impede a criação e edição de políticas IAM customizadas via Terraform), não é possível definir regras de acesso restritas para cada serviço, e por isso todos usam a **LabRole**. Em um deploy de produção, as permissões deveriam ser definidas de acordo com a necessidade de cada serviço.

Na tolerância a falhas, a ingestão de dados utiliza buffering e persistência local via padrão **Outbox** no banco de dados operacional. Falhas de comunicação temporárias com o Firehose ou quedas na execução das Lambdas não interrompem o fluxo de vendas. Da mesma forma, instabilidades nas ferramentas analíticas (Athena, Glue) ou de Machine Learning (SageMaker, Bedrock) degradam apenas a experiência do painel e do chat.

| Falha                 | Comportamento esperado                                                                                                                                     |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Firehose indisponível | Eventos de telemetria ficam acumulados na tabela de Outbox do banco relacional até o restabelecimento do serviço, sem impactar a criação de novos pedidos. |

| Falha                    | Comportamento esperado                                                                                                                                       |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Glue / ETL falha         | As tabelas agregadas Parquet ( <b>marts</b> ) deixam de atualizar temporariamente; o dashboard continua legível com o último estado consolidado.             |
| Athena falha             | O dashboard e o chatbot ficam temporariamente fora do ar por não conseguirem processar consultas, mas a operação transacional principal segue intacta.       |
| Prediction Service falha | O <b>order-service</b> adota um fallback estático de tempo padrão de entrega e cria o pedido normalmente, sem gerar downtime no checkout.                    |
| Bedrock falha            | O chat do assistente retorna um erro controlado informando a impossibilidade de processar linguagem natural, caindo para o motor de fallback determinístico. |

A observabilidade é alcançada com a ajuda do AWS CloudWatch, com a agregação de fluxos de logs estruturados vindos do ECS, instâncias EC2, Lambdas, Step Functions, Glue Jobs e execuções do SageMaker. Esse monitoramento centralizado permite inspecionar falhas de concorrência de queries, erros de sintaxe de views no Athena ou falhas na promoção de modelos pelo pipeline gerenciado de ML diretamente a partir do CloudWatch.

## 7. Implantação e automação

A implantação da A2 estende a abordagem de infraestrutura como código da A1, e todos novos recursos são provisionados junto com a infraestrutura existente.

| Camada          | Recursos provisionados                                                                                                                    |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Ingestão        | Tabela outbox_events, Lambda outbox-publisher, DynamoDB Streams, EventBridge Pipes e Firehose.                                            |
| Data lake       | Bucket S3 com zonas raw, curated e marts; Glue Data Catalog; jobs Glue de transformação.                                                  |
| Dashboard       | Dashboard-service em contêiner Docker sobre EC2 dedicada, lendo Athena/S3 e previsões em S3.                                              |
| Predição        | Prediction-service em ECS/Fargate para serving online de ETA; pipeline gerenciado com SageMaker/Step Functions habilitável para retreino. |
| Conversacional  | Assistant-service em contêiner Docker sobre EC2 dedicada, com Bedrock quando disponível e fallback determinístico.                        |
| Experimentos    | Script scripts/experiments/run_compact_results.py para gerar evidências de carga, analytics, ETA e resiliência.                           |
| Observabilidade | Logs centralizados em CloudWatch para ECS, EC2, Lambdas, Glue e Step Functions.                                                           |

A estratégia de deploy preserva a separação entre build de imagens e provisionamento. As imagens Docker dos novos serviços são publicadas no ECR, enquanto os recursos gerenciados são descritos em IaC. A inicialização do catálogo semântico ocorre como parte do deploy, subindo um arquivo versionado para S3 com definições de negócio e exemplos de perguntas.

## 8. Testes e validação

O objetivo foi medir quatro propriedades centrais da A2: sustentação de carga, persistência de eventos na camada analítica, integração do ETA ao fluxo de criação de pedidos e resiliência operacional quando a predição não está disponível. Os cenários avaliados foram normal, peak e event.

### 8.1. Carga e latência

| Cenário | Criados | Concl. | Falhos | Sucesso | P95 global | P95 /order | SLA     |
|---------|---------|--------|--------|---------|------------|------------|---------|
| Normal  | 204     | 204    | 0      | 100,00% | 259 ms     | 566 ms     | Parcial |
| Peak    | 981     | 970    | 11     | 99,89%  | 2706 ms    | 4754 ms    | Violado |
| Event   | 563     | 280    | 283    | 89,25%  | 10264 ms   | 19611 ms   | Violado |

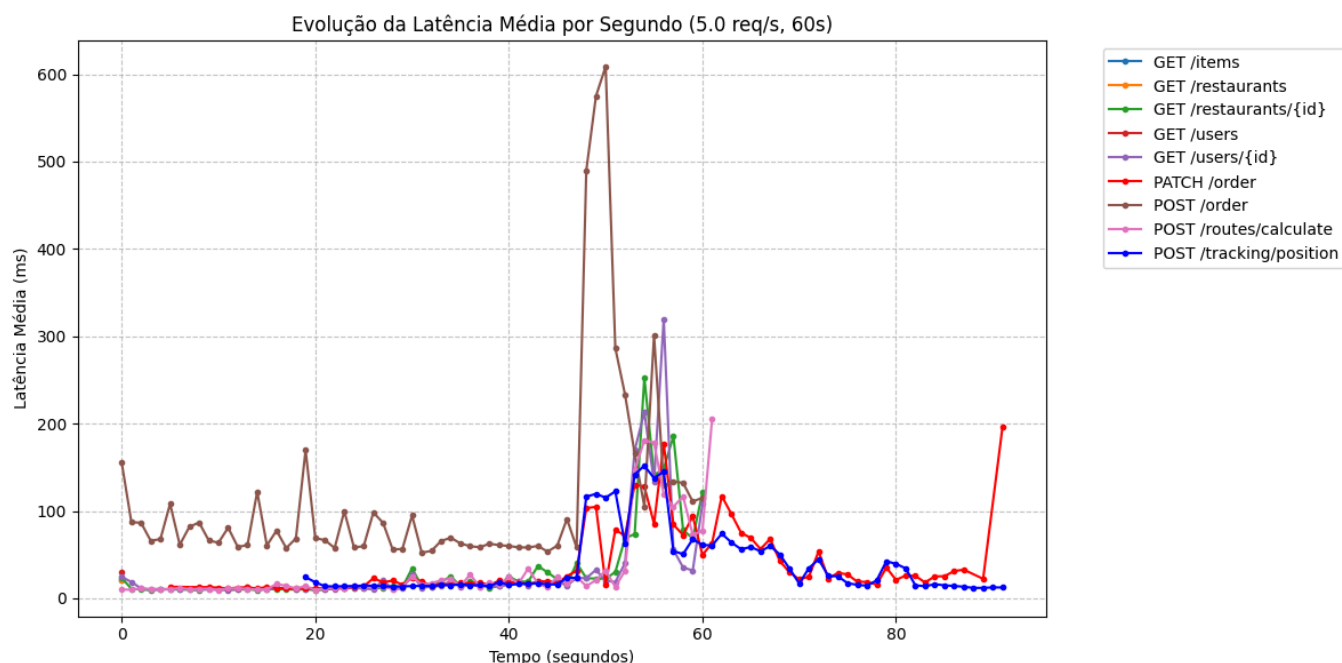


Figura 2: Evolução da latência média por segundo em teste auxiliar de baixa carga, com 5 req/s durante 60 segundos.

No cenário normal, o sistema criou e concluiu 204 pedidos, sem falhas, com taxa de sucesso de 100%. O P95 global ficou em 259 ms, abaixo da meta de 500 ms. Os endpoints de rastreamento e roteamento também ficaram dentro da meta, com P95 de 251 ms para POST /tracking/position e 177 ms para POST /routes/calculate. A única violação relevante nesse cenário foi o endpoint POST /order, com P95 de 566 ms, ligeiramente acima do limite estabelecido.

Nos cenários de maior carga, a plataforma manteve capacidade operacional parcial, mas não atendeu à meta estrita de latência. No cenário peak, foram criados 981 pedidos, dos quais 970 foram concluídos, com taxa de sucesso de 99,89%. Entretanto, o P95 global subiu para 2706 ms, e o P95 de POST /order chegou a 4754 ms. No cenário event, a carga excedeu a capacidade provisionada: foram criados 563 pedidos, mas apenas 280 concluídos, com taxa de sucesso de 89,25%. O gargalo dominante foi POST /order, cujo P95 atingiu 19611 ms.

A figura de baixa carga reforça esse diagnóstico. Durante a maior parte da execução, os endpoints permaneceram estáveis em patamares baixos, mas houve um pico transitório concentrado principalmente em POST /order por volta do intervalo de 50 a 60 segundos. Portanto, os resultados indicam que a otimização reduziu fortemente a latência em operação normal, mas que a criação de pedidos ainda é o principal gargalo nos cenários de pico e evento.

## 8.2. Persistência na camada analítica

| Métrica                            | Valor                   |
|------------------------------------|-------------------------|
| Eventos de criação de pedido       | 2993                    |
| Eventos de transição de estado     | 10330                   |
| Eventos de posição de entregadores | 1753089                 |
| Total de eventos persistidos       | 1785412                 |
| Primeiro evento observado          | 18/06/2026 01:36:43 UTC |
| Último evento observado            | 18/06/2026 02:32:35 UTC |

A camada analítica recebeu eventos das três classes exigidas pelo enunciado: criação de pedidos, transições de estado e posições reportadas. No intervalo experimental, foram persistidos 1.785.412 eventos operacionais, dos quais 2.993 corresponderam à criação de pedidos, 10.330 a transições de estado e 1.753.089 a posições de entregadores. Esse resultado confirma que o pipeline de ingestão foi capaz de registrar os eventos gerados durante a operação sem transformar o data lake em dependência síncrona do caminho crítico.

## 8.3. ETA e resiliência operacional

O prediction-service estava saudável durante a execução, mas o artefato do modelo de ETA não estava carregado (model\_ready=false). Por isso, todas as chamadas de ETA usaram fallback\_constant, retornando uma estimativa fixa de 35 minutos. Esse resultado não valida a acurácia estatística do modelo Random Forest, mas valida a propriedade operacional mais importante: a criação de pedidos não depende criticamente da disponibilidade do modelo.



O teste de resiliência confirmou esse comportamento. Com o serviço de ETA em modo normal, 4 de 5 pedidos foram criados, todos com `eta_source=fallback_constant`; a falha restante ocorreu por indisponibilidade de entregador, não por falha da predição. Em seguida, foi simulada a indisponibilidade do endpoint de ETA por meio de um endpoint inalcançável, e a lógica de fallback retornou corretamente 35 minutos. A criação de pedido continuou possível, confirmando que a camada preditiva enriquece a operação, mas não bloqueia o checkout.

Síntese experimental: validamos a operação normal com 100% de sucesso e P95 global abaixo de 500 ms, comprovou a persistência de eventos na camada analítica e demonstrou resiliência do fluxo de pedidos diante da indisponibilidade do ETA. As principais limitações observadas foram a violação de latência em `POST /order` nos cenários `peak` e `event`, a queda de sucesso no cenário `event` e a ausência de validação estatística do modelo de ETA, já que o serviço operou em fallback durante os testes.

## 9. Análise de custo

A arquitetura prioriza serviços `serverless` e pagamento por uso. A extensão adiciona alguns custos fixos, principalmente as `tasks ECS` da **analytics-api** e da **assistant-api**, mas evita componentes persistentes caros, como um cluster Redshift, um endpoint SageMaker `always-on` para todos os modelos ou um banco vetorial sempre ativo para o chat.

### Estratégia de custo

**Escolha:** `serverless` e `batch` sempre que possível

**Justificativa:** a arquitetura paga mais quando há uso efetivo. Recursos `always-on` foram evitados quando não traziam benefício direto para o Objetivo 3.

As estimativas de preço levam em conta a seguinte distribuição dos cenários ao longo de um mês de 30 dias:

- Cenário de operação normal: 28 dias e 18 horas, ou 690 horas no total;
- Cenário de operação de pico: 1 dia ou 24 horas;
- Cenário de evento: 6 horas.

### 9.1. Custos com ECS

Para calcular os custos de ECS, foram estimadas quantas `tasks` deveriam ser executadas em cada cenário e para cada serviço para aguentar a carga de pedidos sendo feitos. Esse valor foi ponderado pela quantidade de horas de cada evento e multiplicado pelo valor de uma hora com uma `task` ativa. Esse valor de `task/hora` foi calculado considerando \$0,04048/hora de vCPU e \$0,004445/hora de GB de ram.

Vale ressaltar que o serviço **core-api**, por ter natureza de registro de entidades e não ter relação direta com o ciclo de vida de pedidos, foi considerado com execuções constantes ao longo do mês. Cenários em que há aumento no número de registros não foram considerados por não necessariamente se relacionarem com os cenários de aumento no número de entregas.

| Task (Serviço)            | core-api        | routing-ser-vice | tracking-ser-vice | order-service   | prediction-ser-vice |
|---------------------------|-----------------|------------------|-------------------|-----------------|---------------------|
| Configuração (vCPU e RAM) | 2 vCPU / 4 GB   | 2 vCPU / 4 GB    | 0.5 vCPU / 1 GB   | 0.5 vCPU / 1 GB | 2 vCPU / 4 GB       |
| Preço por hora            | \$0.09874       | \$0.09874        | \$0.02469         | \$0.02469       | \$0.09874           |
| Instâncias cenário normal | 2               | 3                | 4                 | 4               | 1                   |
| Instâncias cenário peak   | 2               | 4                | 12                | 12              | 2                   |
| Instâncias cenário evento | 2               | 10               | 30                | 30              | 4                   |
| <b>Preço total do mês</b> | <b>\$142.19</b> | <b>\$219.79</b>  | <b>\$79.68</b>    | <b>\$79.68</b>  | <b>\$75.24</b>      |

**Custo total mensal para os serviços: \$596.58**

### 9.2. Custos com armazenamento

Os custos considerados abaixo levam em conta serviços que persistem informações indefinidamente. Os serviços Amazon RDS e Amazon DynamoDB são utilizados para o funcionamento operacional da plataforma, enquanto o S3 armazena arquivos tanto para funcionalidade operacional quanto para funcionalidade analítica.

#### 9.2.1. Amazon RDS (PostgreSQL)

O custo de computação da instância de banco de dados `db.t3.medium` configurada com `Multi-AZ` para alta disponibilidade é de \$0,136 por hora, o que totaliza \$97,92 para as 720 horas de operação mensal. Para o armazenamento, 100 GB de disco (5x os 20GB de teste) possuem uma tarifa de \$0,23 por GB ao mês, resultando em \$23. Portanto, o custo total mensal consolidado do RDS é de \$120,92.

### 9.2.2. Amazon DynamoDB

Para a realização de testes, o DynamoDB foi configurado no modo On-Demand (PAY\_PER\_REQUEST) para as validações automatizadas do simulador. No entanto, em um cenário de operação constante em produção, o banco seria reconfigurado para o modo de Capacidade Provisionada com Auto Scaling (WCU/RCU dinâmico). Essa arquitetura otimizada provisiona e escala as WCUs de acordo com a demanda real (estimando 600 WCUs em períodos normais, 3.000 WCUs em picos e 12.000 WCUs em eventos). Com a tarifa de \$0,00065 por WCU-hora na região us-east-1, o custo de escrita seria de aproximadamente \$362,70. Somando as leituras e streams, o custo mensal do DynamoDB sob a configuração de produção seria de \$404,55.

### 9.2.3. Amazon S3

Para estimar o S3, adotamos uma baseline fixa inicial de 500 GB de dados ativos no S3 Standard na região us-east-1, cobrindo os arquivos de rotas operacionais, os modelos do SageMaker e o volume analítico quente do último mês. Esse valor foi definido como uma estimativa de 5x o tamanho definido do banco operacional, já que não é trivial calcular o armazenamento total utilizado pelo sistema.

Sob a tarifa de \$0,023 por GB, o custo de armazenamento é de \$11,50. Incluindo \$1 para cobrir requisições de escrita (PUT) e leitura (GET), o custo mensal inicial do S3 é de \$12,50. No entanto, como um Data Lake acumula dados continuamente, o volume do S3 crescerá cerca de 250 GB a cada novo mês de operação normal, fazendo com que o custo de armazenamento subisse de \$12,00/mês para mais de \$80,50/mês (3,5TB) no final do primeiro ano caso tudo permanecesse no S3 Standard. Para sustentação à longo prazo, o projeto precisaria fazer uso de estratégias de cold storage para dados raw e até possivelmente limpeza de dados não agregados da camada operacional após certo tempo.

**Custo total Mensal para armazenamento: \$537,97**

## 9.3. Custos com compute analítico

Os custos com compute analítico levaram em conta serviços que executam processamento direto de informações, sendo eles as funções lambda, os jobs do Glue e o treinamento do modelo de ETA no sagemaker. Também são consideradas as instâncias EC2 que hospedam o dashboard administrativo e gerenciam o chatbot.

| Serviço / Recurso          | Quantidade de Execuções / Estado    | Preço Base (us-east-1)       | Custo Mensal |
|----------------------------|-------------------------------------|------------------------------|--------------|
| Dashboard no EC2           | Constante (720 horas/mês)           | \$0,0208 / hora (t3.small)   | \$14,98      |
| Chat com Assistente no EC2 | Constante (720 horas/mês)           | \$0,0104 / hora (t3.micro)   | \$7,49       |
| Glue Job (build-marts)     | 720 rodadas/mês (~5 min por rodada) | \$0,44 / DPU-hora            | \$26,35      |
| SageMaker (Treino do ETA)  | 30 rodadas/mês (~5 min por rodada)  | \$0,115 / hora (ml.m5.large) | \$0,29       |
| Lambda outbox-publisher    | 43.200 execuções/mês (~300ms)       | Request + GB-segundo         | \$0,06       |
| Lambda ml-callback         | 30 execuções/mês (~3s)              | Request + GB-segundo         | \$0,00 *     |

\* Valor residual/desprezível (menor que \$0,01).

**Custo total de compute para analytics e ML: \$49,17**

## 9.4. Custos com demais serviços

| Serviço                  | Uso Estimado Mensal         | Métrica de Precificação          | Custo Total (USD) |
|--------------------------|-----------------------------|----------------------------------|-------------------|
| Kinesis Data Firehose    | 250 GB de dados brutos      | \$0,029 por GB ingerido          | \$7,25            |
| EventBridge Pipes        | 13,10 milhões de lotes      | \$0,80 por milhão de lotes *     | \$10,48           |
| Amazon Athena            | 5,64 TB de dados escaneados | \$5,00 por TB escaneado          | \$28,20           |
| AWS Step Functions       | 90 transições de estado     | \$25,00 por milhão de transições | \$0,00 **         |
| EventBridge Scheduler    | 30 ativações de schedulers  | \$1,25 por milhão de ativações   | \$0,00 **         |
| Bedrock (Tokens Entrada) | 12,00 milhões de tokens     | \$0,035 por milhão de tokens     | \$0,42            |
| Bedrock (Tokens Saída)   | 1,80 milhões de tokens      | \$0,140 por milhão de tokens     | \$0,25            |

\* Inclui \$0,40/milhão de requisições de stream e \$0,40/milhão de transformações via template de entrada.

\*\* Isento por estar dentro da franquia mensal do Free Tier da AWS.

**Custo total dos demais serviços: \$53,15**

**Custo total mensal dos serviços: \$1236,87**